MOBILE DJ / PRODUCT BLUEPRINT

DOCUMENT 15 OF 19

Video, karaoke, broadcast and Android plugin hosting

A new touch-first performance instrument.
Professional audio, liquid glass and mechanical craft.



DESIGN HANDOFF BASELINE - PLANNING ONLY

29 October 2026 | Crystal Current selected; S26 Ultra test target

Independent of the previous Mixline preview. Targets are not measured results.

Version: 1.0 **Date:** 29 October 2026 **Status:** Design handoff baseline; selected product scope with unverified implementation contracts and certification targets **Owners:** Audio lead, Android media lead, integrations lead, security lead and device QA lead

1. Scope and completion rule

The user selected every enumerated professional module except lighting. Video mixing, karaoke, broadcasting and actual third-party plugin hosting therefore belong in the target complete personal release. They cannot be silently deferred or replaced by decorative controls. Lighting/DMX is excluded. External DJ-controller integration remains excluded; internal events used by an instrument plugin do not automatically create a controller-integration requirement.

The delivery order is now local Android first, with AWS expansion later. These selected modules are designed around the phone's engine and measured local resources, with no required AWS worker or home-lab server. Broadcasting still needs an explicitly chosen network destination; optional AI and music acquisition can need connectivity. Those independent network features do not turn cloud audio processing or a self-hosted service into a prerequisite. The future AWS planning target of about US$35/month excludes ParDay AI-provider charges and makes no promise that a cloud worker can solve an unqualified local video/plugin workload.

Android is the first platform, and Galaxy S26 Ultra is the confirmed first test device. Its software is reported up to date; the exact installed build and accessory route remain uninspected. This selection does not certify any concurrent workload. C Crystal Current supplies the visual family for all specialist workspaces. USB audio interfaces and splitters are permitted. Private practice and personal gatherings are the initial use context, but starting an Internet broadcast changes the audience and source permissions. The broadcaster must resolve those permissions separately. A playable local video or streamed song is not automatically cleared for lyrics display, recording, synchronization or broadcast; Documents 07, 11 and 14 own the source policy.

This document defines contracts before choosing exact video containers/codecs, karaoke formats, broadcast protocols or plugin formats. The targets below are proposed acceptance thresholds, not measured capabilities. A device profile can limit concurrent workloads only when that limitation is explicit. If no supported configuration can satisfy a selected feature, return the gap to the user before changing the completion promise.

2. Shared timing and resource architecture

The native audio engine's compensated sample clock remains authoritative. A media-time mapper supplies each deck's source position, transport generation, tempo map, seek epoch and presentation target. Video, timed lyrics, network timestamps and plugin automation consume that mapping. UI animation, a decoder's wall clock or a network receiver must not change musical tempo to make pictures look smoother.

Use bounded queues between demux, audio decoding, video decoding, composition, encoding and network transfer. No codec configuration, GPU wait, Binder transaction, plugin discovery, network operation or lyrics parsing occurs inside the device callback. Declare memory budgets and backpressure behavior for every queue. A slow video or network branch must not block master or cue audio.

Android's MediaCodec documentation recommends Surface-based raw video paths for efficiency, while supported codec behavior depends on platform and device capabilities. [1,2] Treat those APIs as building blocks. A successful single-video demonstration does not certify simultaneous decoding, compositing, recording and stem separation on a hot phone.

3. Video mixing contract

Begin with authorized unprotected local video and independently admitted provider video. Store audio and video track references against the same media identity with offsets, duration, rotation, aspect ratio, color characteristics and variable-frame-rate timestamps. Reject or explain unsupported streams; do not mislabel a file playable because its extension is familiar.

The proposed Android path is off-thread demux plus bounded MediaCodec decoders into Surfaces, a GPU compositor, and separate preview/program outputs. Decode only the admitted number of simultaneously needed video streams. A four-audio-deck layout does not imply four full-resolution video decoders can always run. Cache thumbnails separately and release unused decoder resources outside an active transition.

Each deck selects video-follow-audio, independent visual content, or audio-only. In linked mode, video follows the engine's source position and tempo; key changes alone do not change picture speed. A loop repeats the corresponding video interval. On a seek, invalidate old frames and resume the new epoch; never flash stale imagery from the previous track. Exact reverse/scratch video is a separate certification case because inter-frame compression may prevent immediate arbitrary-frame decoding. Until certified, freeze or replace that deck's visual according to a visible selected policy while preserving audio scratch behavior.

The program compositor supports a defined transition set, beginning with cut, dissolve and linked/unlinked video crossfade. Keep visual crossfader state separate from the audio crossfader when unlinked. Audio-only tracks use a chosen licensed still, restrained app visual or black; no invented album cover or unrelated video is silently substituted. Unsupported HDR-to-SDR conversion must be rejected or explicitly handled rather than producing incorrect washed-out output.

External display is a capability, not a guarantee for every USB-C port. Android Presentation supplies a secondary-display mechanism; actual hardware, display route and lifecycle must be tested. [3] Show a clean program surface while controls remain on the device. An external display disconnect must not expose private UI, cue controls, stream keys or the singer's private notes. A reconnect requires a valid display configuration. Wireless display latency needs its own profile and must not be advertised as equivalent to a verified wired route.

4. Karaoke and microphone workflow

Karaoke is a real performance mode with a singer queue, selected song/version, key/tempo settings, lyric/video presentation, microphone routing and an operator-controlled next singer. Keep stage names and queue notes local by default. Reordering or removing a waiting singer must not replace the active song. Include a clear pause, skip and return-to-DJ flow with recoverable queue state.

Admit licensed local video karaoke, an approved timed-text format, and a CD+G route only after parser, compatibility and rights review. WebVTT supplies a standardized timed-text basis; paired audio/CDG and archive variants exist in established open-source karaoke software, but borrowing that implementation requires its own license review. [4,5] The exact shipped subset remains a feasibility decision. Do not promise every LRC dialect, proprietary karaoke package or legacy disc format.

Store lyric timing in source-media time so the same mapper follows tempo, seeks and loops. Preserve line breaks, readable contrast, language shaping and safe display margins. Word highlighting requires word-level timing; do not fabricate precision from a line-timed file. Provide an adjustable measured display offset without changing the singer's audio. No automatic web lyric scraping or generated “official lyrics” is required. User-authored text and separately licensed lyrics are valid inputs.

Microphone features include selected input/channel, gain, meter, mute, monitoring choice, optional EQ/compression and a controlled mix/broadcast assignment. Make feedback risk understandable at routing time. An automatic ducking option uses an explicit threshold/envelope; manual talk-over remains available.

Do not route microphone audio into ParDay merely because the karaoke microphone is enabled. If a headset/USB device changes, apply the audio route-loss rules before enabling monitoring again.

5. Broadcast architecture and operation

Broadcasting consumes an explicitly selected eligible program bus, normally master plus chosen microphone/video. Cue audio and private assistant speech are excluded by default. Before Start, show endpoint/account, audience or destination, selected buses, source eligibility and selected quality profile. Starting/stopping is a deliberate user action; ParDay can prepare a proposal but cannot silently begin transmission.

Use a bounded audio/video capture queue, dedicated encoder workers, mux/protocol adapter and independent network queue. Media timestamps derive from the engine clock and declared video mapping. The encoder can report delay without feeding timing corrections into the live mix. Choose a supported protocol after target-service testing: RTMP/RTMPS or enhanced RTMP and SRT are candidates with published projects/specifications, not a blanket compatibility promise. [6,7] Container, codec, encryption, authentication and receiver support are separate decisions.

Expose connection state, encoded rate, queue delay, dropped frames, reconnect attempts and whether the destination acknowledged reception. “Connected” does not prove the audience is hearing the correct mix. Provide a test destination/receiver check before a real broadcast. Store stream keys securely and redact them from URLs, logs, screenshots and diagnostic exports. Never send them to a language model.

On congestion, use a declared bounded policy: reduce video bitrate or frames within the selected profile, reconnect, or stop transmission with an alert. Do not accumulate unlimited latency, silently send a delayed entire set, or stall local audio. Preserve temporal continuity when dropping video; request a clean keyframe/resynchronization where the format requires it. Network switching and authentication failures must be tested.

Local fallback recording is a separate user-selected, source-permitted option. It must already have storage and recording permission, and cannot be enabled merely because the stream failed. Save recoverable segments and mark broadcast gaps honestly. Stopping broadcast releases transmission promptly without muting the room mix or discarding a valid independent recording.

6. Actual Android third-party plugin hosting

Built-in effects are part of the engine, but they do not fulfill this requirement. Completion requires a documented host interface and at least one independently authored compatible plugin operating through it; the proposed validation corpus includes an effect and an instrument. Users need discovery, eligibility, load/unload, parameters, presets, state recall, automation, bypass and clear compatibility information.

Desktop VST/AU binaries must not be assumed compatible with Android CPU, operating system or packaging. The current VST3 SDK's supported-platform table does not list Android. AAP, Audio Plugins For Android, is a concrete Android-oriented candidate using separate host/plugin applications and shared-memory/IPC mechanisms; its own project warns that its APIs are immature. [8,9] Benchmark an exact pinned version rather than treating a repository's existence as production readiness.

Compare two candidates at the feasibility gate: an AAP adapter with independently packaged compatible plugin applications, and a small documented app-native plugin interface with curated third-party modules delivered through the app's approved package/update channel. The second option can support real third-party authorship, but only if independently authored modules use that interface; renaming internal effects as plugins does not count. A closed compatibility list is acceptable only when clearly stated and accepted as the supported host scope.

The plugin contract declares ID/vendor/version, ABI and CPU architecture, inputs/outputs, sample formats, supported rates/block sizes, parameter IDs, state schema, maximum memory, measured CPU budget,

latency, tail length and required capabilities. Prepare/load/state parsing happens off the callback. Processing must meet a fixed deadline; latency changes require negotiated graph reconstruction outside active use. Presets cannot contain executable code or arbitrary filesystem/network instructions.

Prefer a process boundary for untrusted third-party code. An asynchronous worker/shared-memory design must supply ready blocks at a fixed declared latency; the audio callback never waits for IPC. Measure round-trip delay, jitter, crash behavior and CPU cost. An in-process trusted plugin runs with the host's privileges and can crash it; do not label that arrangement sandboxed. State clearly which isolation model each admitted class uses.

Define fault behavior before activation: latency-compensated dry bypass for effects where valid, or a short controlled instrument mute; alert the user and preserve unaffected decks. A plugin timeout must not trigger endless repetition of a stale block. Session restore checks plugin identity/state version and offers an unavailable-plugin placeholder rather than destroying the saved setup. Missing plugins cannot silently become a different effect.

Google Play prohibits downloading executable dex/JAR/native .so code from sources outside its update mechanism. Code signing alone does not remove that restriction. [10,11] For a Play release, use packaged/delivered approved modules or separately installed compatible apps only after reviewing the actual distribution design. No arbitrary remote-native-plugin marketplace is assumed. Personal non-Play distribution is now the selected milestone. That removes public-store submission from the immediate release gates while preserving installation, signing, component-licence and isolation requirements. Any later Play release needs a new distribution review; Document 17 covers optional community-source sharing. Prefer compatible open source, while checking exact code, model, SDK, trademark and redistribution obligations independently.

7. Testable module requirements

Each row has a stable ID for traceability into the global register. All are proposed acceptance contracts within the selected feature scope.

ID	Required behavior	Acceptance evidence
AV-001	Video follows compensated audio media time.	Measured flash/click fixtures across tempo, seek and loop; no accumulating drift.
AV-002	Video sources and generations remain distinct.	Rapid load/cancel/seek never displays frames from an obsolete assignment.
AV-003	Deck video modes and program transitions work.	Linked/unlinked transitions, audio-only fallback and unsupported stream tests.
AV-004	External program output is separate and recoverable.	Wired display attach/remove/reconnect; no private UI or cue leakage.
AV-005	Karaoke queue and timed presentation remain coherent.	Singer reorder, skip, pause, transpose, tempo and session restore fixtures.
AV-006	Mic, ducking and bus assignment are deliberate.	Physical input/monitor test; route loss, clipping and consent checks.
AV-007	Broadcast starts/stops only by authorized action.	Correct endpoint/bus preview; cancellation and unauthorized action rejected.

ID	Required behavior	Acceptance evidence
AV-008	Network/encoder failures cannot block local audio.	Loss, congestion, reconnect, wrong key and worker crash during warm playback.
AV-009	Broadcast/recording enforce source restrictions.	Ineligible next track resolved before transmission; partial output labelled.
AV-010	A real independently authored plugin is hosted.	Compatible effect/instrument corpus loads, processes, automates and recalls state.
AV-011	Plugin faults and latency are bounded.	Timeout/crash/state corruption/latency-change tests preserve unaffected audio.
AV-012	Plugin distribution and isolation claims are accurate.	Package provenance, license, store-policy and process-boundary review.
AV-013	Concurrent module promises match measured profiles.	Two-hour repeated warm runs and actual receiver/display observations.
AV-014	Advanced work never enters the device callback.	Instrumented allocation/block/I/O/IPC checks and deadline traces.

8. Device workloads and release gates

Start with these candidate profiles, then publish the exact tested device, OS, audio route, codecs, resolutions, plugin builds, model and accessory configuration. Proposed video timing target: absolute picture/audio offset within 80ms for 95% of observations and no growing drift over 30 minutes; calibrate wired and wireless displays independently. This is a measurable initial target, not an established professional standard or a guarantee.

Profile	Candidate sustained workload	Required evidence
AV-P0	Two audio decks, one 720p30 video, timed lyrics, mic and external-display candidate.	Correct routing/timing; declared decoder and display limits.
AV-P1	Two simultaneously decoded 720p30 deck videos, GPU crossfade, mic and permitted recording.	Two-hour warm playback plus seek/transition stress.
AV-P2	Four audio decks, declared video-deck count, two admitted plugins, sampler/FX and broadcast.	End-to-end receiver capture, loss recovery and plugin failure isolation.
AV-P3	Selected stems profile plus video/broadcast/plugins and ParDay activity.	Combined thermal/memory/callback evidence; independently passing modules is insufficient.

Record queue ceilings, callback deadlines, memory high-water marks, video loss, thermal state, total plugin latency, broadcast delay and battery/power conditions. Before enabling a workload, check admission against the current device profile. Resource pressure can pause optional preparation and simplify decorative visuals. It must not silently alter active audio quality, timing, routing or stem alignment. A profile that cannot sustain its promised combined workload fails certification.

G1 adds focused codec/display/plugin feasibility; G2 establishes the core audio slice and clock interfaces; dedicated working slices for these selected modules must precede their G4 completion. G4 expands the full controls and workloads; G6 requires AV-001-014 evidence, asset/source rights and failure recovery. G7 complete personal-release acceptance includes all selected modules; it does not require the later optional AWS extension. If evidence supports only a smaller release, present the exact missing capabilities and obtain a product decision. No such reduction has been approved here.

Sources checked 29 October 2026

1. Android MediaCodec.
2. Android supported media formats.
3. Android Presentation.
4. W3C WebVTT.
5. OpenKJ source and format documentation, implementation license requires independent review.
6. Veovera enhanced RTMP specification project.
7. Haivision SRT project.
8. Steinberg VST3 SDK supported platforms.
9. AAP Android audio plugin framework, exact version/stability to be evaluated.
10. Google Play Device and Network Abuse policy.
11. Android dynamic-code-loading risks.